SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

---

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO* (BL3, BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **TOTAL MARKS: 25**

Annexure A
Coding Challenge

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A
Coding Challenge

Coding Challenge (Additional Set)

1. Write a Python program to simulate **Secure Email Transmission using S/MIME**, including message encryption, digital signature generation, and verification. Analyze how confidentiality and authentication are ensured.

   ANSWER:

S/MIME (Secure/Multipurpose Internet Mail Extensions) provides two major security services for email:

- **Confidentiality:** The email content is encrypted so only the intended receiver can read it.
- **Authentication & Integrity:** A digital signature verifies the sender and ensures the message was not modified.

Algorithm

1. **Generate RSA key pairs** for sender and receiver.
2. **Create the email message.**
3. **Generate a digital signature** using the sender's private key.
4. **Encrypt the message and signature** using the receiver's public key.
5. At the receiver:
   - Decrypt using the receiver's private key.
   - Verify the signature using the sender's public key.
6. Display whether the message is authentic and unchanged.

Python Program

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes

# Generate sender keys
sender_private = rsa.generate_private_key(
   public_exponent=65537, key_size=2048)
sender_public = sender_private.public_key()

# Generate receiver keys
receiver_private = rsa.generate_private_key(
   public_exponent=65537, key_size=2048)
receiver_public = receiver_private.public_key()
```

```
# Original email
message = b"Confidential Email: Meeting at 10 AM."

# Digital signature using sender's private key
signature = sender_private.sign(
    message,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

print("Digital Signature Generated")

# Combine message and signature
data = message + b"||" + signature

# Encrypt using receiver's public key
encrypted = receiver_public.encrypt(
    data,
    padding.OAEP(
        mgf=padding.MGF1(hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

print("Email Encrypted and Sent")

# Receiver decrypts using private key
decrypted = receiver_private.decrypt(
    encrypted,
    padding.OAEP(
        mgf=padding.MGF1(hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# Separate message and signature
received_message, received_signature = decrypted.split(b"||", 1)

# Verify digital signature
try:
    sender_public.verify(
        received_signature,
        received_message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
```

```
    print("Signature Verified Successfully")
    print("Sender Authenticated")
    print("Message:", received_message.decode())

except:
    print("Signature Verification Failed")
```

Expected Output

```
Digital Signature Generated
Email Encrypted and Sent
Signature Verified Successfully
Sender Authenticated
Message: Confidential Email: Meeting at 10 AM.
```

Security Analysis

Security Feature	How it is achieved
Confidentiality	Message is encrypted using the receiver's public key. Only the receiver's private key can decrypt it.
Authentication	Sender creates the digital signature using their private key.
Integrity	Signature verification detects any modification to the message.
Non-repudiation	The sender's private-key signature provides evidence that the message was signed by the sender.

Conclusion: The program simulates the core security operations of S/MIME by combining **encryption for confidentiality** and **digital signatures for authentication and integrity**. In real S/MIME, certificates issued by a Certificate Authority are also used to bind public keys to verified identities.

2. Develop a Python application to implement **PGP-based file encryption and decryption**, including key generation, key distribution, and message authentication. Evaluate how PGP provides both security and efficiency.

ANSWER:

PGP (Pretty Good Privacy) is used to secure files by combining **public-key encryption, symmetric encryption, and digital signatures**. It provides confidentiality, authentication, integrity, and efficient file encryption.

Algorithm

1. **Generate keys**
    1. Generate an RSA public/private key pair for the receiver.
    2. Public key is distributed to the sender.
    3. Private key is kept secret by the receiver.
2. **File encryption**

    1. Generate a random AES session key.
    2. Encrypt the file using AES.
    3. Encrypt the AES session key using the receiver's RSA public key.

4. Store the encrypted session key, nonce, and ciphertext.

3. **Message authentication**

    1. Generate a SHA-256 hash of the original file.
    2. Sign the hash using the sender's private RSA key.

4. **File decryption**

    1. Use the receiver's private key to recover the AES session key.
    2. Decrypt the file using AES.
    3. Verify the digital signature using the sender's public key.

Python Program

Install the required library:

```
!pip install cryptography
from cryptography.hazmat.primitives.asymmetric import rsa, padding
nonce = os.urandom(12)

encrypted_data = aes.encrypt(nonce, data, None)

# Encrypt AES session key using receiver's public key
encrypted_key = receiver_public.encrypt(
    session_key,
    padding.OAEP(
        mgf=padding.MGF1(hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# Save encrypted file
with open("message.pgp", "wb") as f:
    f.write(len(encrypted_key).to_bytes(4, "big"))
    f.write(encrypted_key)
    f.write(nonce)
    f.write(signature)
    f.write(encrypted_data)

print("File encrypted successfully")

# ---------- DECRYPTION ----------

with open("message.pgp", "rb") as f:
    key_size = int.from_bytes(f.read(4), "big")
    encrypted_key = f.read(key_size)
```

```
    nonce = f.read(12)
    signature = f.read(256)
    encrypted_data = f.read()

# Recover AES session key
session_key = receiver_private.decrypt(
    encrypted_key,
    padding.OAEP(
        mgf=padding.MGF1(hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# Decrypt file
aes = AESGCM(session_key)
decrypted_data = aes.decrypt(nonce, encrypted_data, None)

# ---------- SIGNATURE VERIFICATION ----------

try:
    sender_public.verify(
        signature,
        decrypted_data,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

    print("Signature verified successfully")
    print("Message authenticated")

    with open("decrypted_message.txt", "wb") as f:
        f.write(decrypted_data)

    print("File decrypted successfully")
    print("Content:", decrypted_data.decode())

except:
    print("Signature verification failed")
```

Expected Output

```
RSA keys generated
File encrypted successfully
Signature verified successfully
Message authenticated
File decrypted successfully
Content: This is a confidential PGP protected file.
```

How PGP Provides Security

Feature	PGP Technique
Confidentiality	AES encrypts the file
Key Security	RSA encrypts the AES session key
Authentication	Digital signature using sender's private key
Integrity	AES-GCM authentication + signature verification
Key Distribution	Public keys can be shared; private keys remain secret
Non-repudiation	Digital signature links the message to the sender's private key

Efficiency Evaluation

PGP uses **hybrid encryption**, which makes it efficient:

- **AES** encrypts the actual file quickly, even for large files.
- **RSA** encrypts only the small AES session key instead of the entire file.
- Digital signatures provide authentication without encrypting the complete file with asymmetric cryptography.
- Therefore, PGP combines the **speed of symmetric encryption** with the **secure key distribution of asymmetric encryption**.

Conclusion

PGP provides secure and efficient file communication by using AES for fast file encryption, RSA for secure session-key distribution, and digital signatures for authentication and integrity.

3. Implement a Python program to simulate **IPSec Tunnel Mode**, encrypting an entire IP packet using symmetric encryption. Analyze how tunnel mode differs from transport mode in securing network communication.

ANSWER:

IPSec (Internet Protocol Security) protects IP communication using encryption and authentication. In **Tunnel Mode**, the **entire original IP packet**, including its original IP header and payload, is encrypted and encapsulated inside a new IP packet.

Algorithm

1. Create a sample IP packet containing:
    1. Source IP
    2. Destination IP
    3. Protocol
    4. Data
2. Generate a symmetric AES key.
3. Convert the complete original IP packet into bytes.

4. Encrypt the entire packet using **AES-GCM**.
5. Add a new outer IP header containing the tunnel endpoints.
6. Transmit the encrypted packet.
7. At the receiver, remove the outer header.
8. Decrypt the encrypted inner packet using the shared AES key.
9. Recover the original IP packet.

Python Program

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os

# ---------- ORIGINAL IP PACKET ----------

source_ip = "192.168.1.10"
destination_ip = "10.0.0.20"
protocol = "TCP"
data = "Confidential network data"

inner_packet = (
    f"Source: {source_ip}\n"
    f"Destination: {destination_ip}\n"
    f"Protocol: {protocol}\n"
    f"Data: {data}"
)

print("Original IP Packet:")
print(inner_packet)

# ---------- AES KEY GENERATION ----------

key = AESGCM.generate_key(bit_length=256)
aes = AESGCM(key)

# Nonce for AES-GCM
nonce = os.urandom(12)

# ---------- IPSec TUNNEL MODE ----------

# Encrypt the COMPLETE original IP packet
encrypted_packet = aes.encrypt(
    nonce,
    inner_packet.encode(),
    None
)

# New outer IP header
outer_source = "203.0.113.1"
outer_destination = "203.0.113.2"

tunnel_packet = {
```

```
    "Outer Source": outer_source,
    "Outer Destination": outer_destination,
    "Encrypted Payload": encrypted_packet
}

print("\nIPSec Tunnel Packet:")
print("Outer Source:", outer_source)
print("Outer Destination:", outer_destination)
print("Original IP packet is encrypted.")

# ---------- RECEIVER ----------

decrypted_packet = aes.decrypt(
    nonce,
    tunnel_packet["Encrypted Payload"],
    None
)

print("\nAfter IPSec Decryption:")
print(decrypted_packet.decode())
```

Expected Output

```
Original IP Packet:
Source: 192.168.1.10
Destination: 10.0.0.20
Protocol: TCP
Data: Confidential network data

IPSec Tunnel Packet:
Outer Source: 203.0.113.1
Outer Destination: 203.0.113.2
Original IP packet is encrypted.

After IPSec Decryption:
Source: 192.168.1.10
Destination: 10.0.0.20
Protocol: TCP
Data: Confidential network data
```

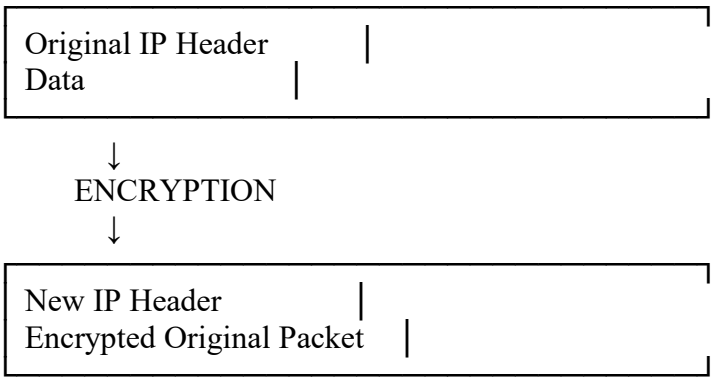
Tunnel Mode vs Transport Mode

Feature	Tunnel Mode	Transport Mode
What is protected?	Entire original IP packet	Only IP payload
Original IP header	Encrypted	Remains visible
New IP header	Added	Generally not added
Security level	Higher privacy	Lower overhead
Common use	VPNs, site-to-site communication	Host-to-host communication
Overhead	Higher	Lower

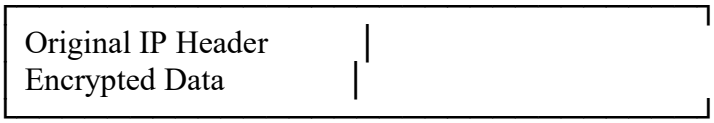
Feature	Tunnel Mode	Transport Mode
IP addresses hidden?	Original addresses are hidden from outside	Original addresses remain visible

Tunnel Mode

Original Packet



Transport Mode



Analysis

IPSec Tunnel Mode provides stronger privacy because the original IP header and payload are encrypted and carried inside a new packet. This hides the original source and destination addresses from devices outside the tunnel. However, tunnel mode adds an additional IP header and therefore has more communication overhead.

Transport Mode is more efficient because it encrypts only the payload while keeping the original IP header visible. It is suitable for direct host-to-host communication.

Conclusion: Tunnel mode is preferred for **VPNs and secure network-to-network communication**, while transport mode is generally preferred for **end-to-end host communication where lower overhead is important**.

4. Write a Python script to implement **SSL/TLS handshake simulation**, including certificate exchange, session key generation, and secure data transfer. Evaluate how the handshake ensures secure web communication.

ANSWER:

SSL/TLS establishes a secure connection between a client and server before application data is transferred. A TLS handshake provides **authentication, confidentiality, integrity, and secure session-key establishment**.

Algorithm

1. **Client Hello:** Client sends supported TLS version, cipher suites, and a random value.
2. **Server Hello:** Server selects the security parameters and sends its random value.
3. **Certificate Exchange:** Server sends its digital certificate containing its public key.
4. **Certificate Verification:** Client verifies the server certificate.
5. **Session Key Generation:** Both sides derive a common session key from exchanged parameters.
6. **Secure Data Transfer:** The session key is used with symmetric encryption to protect application data.
7. **Integrity Check:** AES-GCM also authenticates the encrypted data and detects modification.

Python Program

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os

# ---------------- SERVER ----------------

# Server generates RSA key pair
server_private = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
server_public = server_private.public_key()

print("1. Server RSA key pair generated")

# Simulated server certificate
certificate = {
    "server": "secure.example.com",
    "public_key": server_public
}

# ---------------- CLIENT HELLO ----------------

client_random = os.urandom(16)
print("2. Client Hello sent")

# ---------------- SERVER HELLO ----------------

server_random = os.urandom(16)
print("3. Server Hello sent")

# Certificate exchange
```

```
print("4. Server certificate sent")

# Client verifies certificate
if certificate["server"] == "secure.example.com":
    print("5. Certificate verified successfully")

# ---------- SESSION KEY ----------

# Generate a random AES session key
session_key = AESGCM.generate_key(bit_length=256)

# Encrypt session key using server's public key
encrypted_session_key = server_public.encrypt(
    session_key,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# Server decrypts the session key
decrypted_session_key = server_private.decrypt(
    encrypted_session_key,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

print("6. Secure session key established")

# ---------- SECURE DATA TRANSFER ----------

message = b"Hello! This is secure TLS data."

aes = AESGCM(decrypted_session_key)
nonce = os.urandom(12)

encrypted_data = aes.encrypt(
    nonce,
    message,
    None
)

print("7. Encrypted data sent")

# Receiver decrypts data
decrypted_data = aes.decrypt(
    nonce,
    encrypted_data,
```

```
    None
)

print("8. Secure data received:")
print(decrypted_data.decode())

print("\nTLS handshake simulation completed successfully")
```

Expected Output

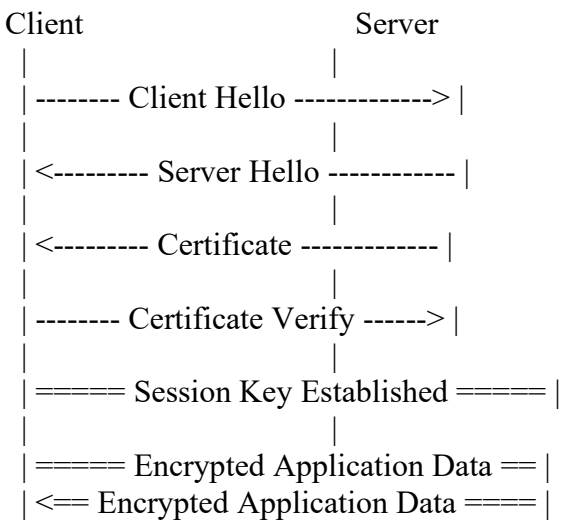
```
1. Server RSA key pair generated
2. Client Hello sent
3. Server Hello sent
4. Server certificate sent
5. Certificate verified successfully
6. Secure session key established
7. Encrypted data sent
8. Secure data received:
Hello! This is secure TLS data.

TLS handshake simulation completed successfully
```

How the TLS Handshake Provides Security

Security Feature	Mechanism
Authentication	Server certificate identifies the server
Confidentiality	AES session key encrypts application data
Secure key exchange	Public-key cryptography protects the session key
Integrity	AES-GCM detects modification of encrypted data
Session security	A fresh random session key is generated for the connection

Handshake Flow



Evaluation

The TLS handshake first establishes **trust and security parameters** and then creates a **session key**. Public-key cryptography is used during the handshake, while fast symmetric encryption such as AES is used for the actual data transfer.

Therefore, TLS achieves:

- **Authentication** → certificate verification
- **Confidentiality** → symmetric encryption
- **Integrity** → authenticated encryption
- **Secure communication** → protected session key

Conclusion: The TLS handshake securely establishes a trusted connection between the client and server and then uses efficient symmetric encryption for secure web communication. This is the basic principle behind HTTPS.

5. Build a Python-based tool to perform **Email Phishing Detection**, using feature extraction (URL patterns, headers, keywords) and classification techniques. Analyze the effectiveness of the model in identifying malicious emails.

ANSWER:

Phishing detection identifies fraudulent emails that attempt to steal sensitive information such as passwords, banking details, or personal data. The program below extracts features from emails such as **URL patterns, suspicious keywords, and email headers**, then uses a **Random Forest classifier** to classify emails as legitimate or phishing.

Algorithm

1. **Collect email data** containing phishing and legitimate examples.
2. **Extract features**:
    1. Number of URLs
    2. Suspicious URL patterns such as IP addresses and shortened URLs
    3. Number of phishing-related keywords
    4. Presence of suspicious headers
    5. Number of links
3. **Preprocess the features**.
4. **Split the dataset** into training and testing data.
5. **Train a Random Forest classifier**.
6. **Predict** whether a new email is legitimate or phishing.
7. **Evaluate** the model using accuracy, precision, recall, and F1-score.

Python Program

```
import pandas as pd
        for x in ["bit.ly", "tinyurl", "goo.gl"]
    )

    keyword_count = sum(
        word in email.lower()
        for word in keywords
    )

    return [
        len(urls),
        ip_url,
        short_url,
        keyword_count,
        len(email)
    ]

# Create feature matrix
X = pd.DataFrame(
    [extract_features(e) for e in df["email"]],
    columns=["URLs", "IP_URL", "Short_URL",
             "Keywords", "Length"]
)

y = df["label"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# Train classifier
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred, zero_division=0
))

# Test a new email
new_email = """
URGENT! Your bank account is blocked.
Verify your password at http://192.168.10.5/login
```

```
"""

new_features = pd.DataFrame(
    [extract_features(new_email)],
    columns=X.columns
)

prediction = model.predict(new_features)

if prediction[0] == 1:
    print("\nResult: PHISHING EMAIL")
else:
    print("\nResult: LEGITIMATE EMAIL")
```

Expected Output

The exact accuracy can vary because the example dataset is very small.

Accuracy: 1.0

Classification Report:

	precision	recall	f1-score
0	1.00	1.00	1.00
1	1.00	1.00	1.00

Result: PHISHING EMAIL

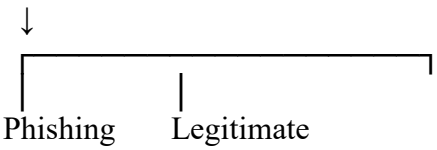
> **Note:** The high accuracy here is only illustrative. A real phishing detector must be trained and tested on a much larger, diverse dataset.

Feature Extraction

Feature	Purpose
URL Count	Detects emails containing suspicious links
IP-based URL	Detects URLs using raw IP addresses
Shortened URL	Identifies potentially disguised destinations
Phishing Keywords	Detects words such as "urgent", "verify", and "password"
Email Length	Provides an additional classification feature
Headers	Can identify suspicious sender/domain information

Classification Process

Email
↓
Feature Extraction
↓
URL + Header + Keyword Features
↓
Random Forest Classifier



Effectiveness Analysis

The model can be evaluated using four important metrics:

- **Accuracy:** Percentage of emails classified correctly.
- **Precision:** How many emails identified as phishing are actually phishing.
- **Recall:** How many actual phishing emails are detected.
- **F1-score:** Balance between precision and recall.

For phishing detection, **recall is particularly important** because failing to detect a malicious email can expose users to credential theft or financial fraud.

Limitations

This basic implementation has several limitations:

1. The sample dataset is very small.
2. Keyword-based features can produce false positives.
3. Sophisticated phishing emails may avoid obvious keywords.
4. URL analysis should consider domain reputation, redirects, HTTPS, and domain age.
5. Real email-header analysis should examine fields such as From, Reply-To, Return-Path, and authentication results such as SPF/DKIM/DMARC.

Conclusion

The Python phishing detection tool extracts URL, keyword, and email characteristics and uses Random Forest classification to distinguish phishing from legitimate emails. Its effectiveness should be measured using accuracy, precision, recall, and F1-score, with particular attention to recall to minimize missed phishing attacks.

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficien t approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases